

FPGA ASSESSMENT 3

Anonymous public copy

Personal and institutional identifiers removed.

Contents

1	Program 1 : S-R Flip Flop	3
1.1	Verilog Code	3
1.2	Testbench	3
1.3	Simulation Output	4
2	Program 2 : J-K Flip Flop	5
2.1	Verilog Code	5
2.2	Testbench	5
2.3	Simulation Output	6
3	Program 3 : T Flip Flop	7
3.1	Verilog Code	7
3.2	Testbench	7
3.3	Simulation Output	8
4	Program 4 : D Flip Flop	9
4.1	Verilog Code	9
4.2	Testbench	9
4.3	Simulation Output	10
5	Program 5 : SISO	11
5.1	Verilog Code	11
5.2	Testbench	11
5.3	Simulation Output	12
6	Program 6 : SIPO	13
6.1	Verilog Code	13
6.2	Testbench	13
6.3	Simulation Output	14
7	Program 7 : PISO	15
7.1	Verilog Code	15
7.2	Testbench	15
7.3	Simulation Output	16
8	Program 8 : PIPO	17
8.1	Verilog Code	17
8.2	Testbench	17
8.3	Simulation Output	18
9	Program 9 : Bidirectional Shift Register	19
9.1	Verilog Code	19
9.2	Testbench	19
9.3	Simulation Output	20

10 Program 10 : Universal Shift Register	21
10.1 Verilog Code	21
10.2 Testbench	21
10.3 Simulation Output	23
11 Program 11 : Up Counter	24
11.1 Verilog Code	24
11.2 Testbench	24
11.3 Simulation Output	25
12 Program 12 : Down Counter	26
12.1 Verilog Code	26
12.2 Testbench	26
12.3 Simulation Output	27
13 Program 13 : Up-Down Counter	28
13.1 Verilog Code	28
13.2 Testbench	28
13.3 Simulation Output	29
14 Program 14 : MOD - 10 Counter	30
14.1 Verilog Code	30
14.2 Testbench	30
14.3 Simulation Output	31
15 Program 16 : Ring Counter	32
15.1 Verilog Code	32
15.2 Testbench	32
15.3 Simulation Output	33
16 Program 16 : Johnson Counter	34
16.1 Verilog Code	34
16.2 Testbench	34
16.3 Simulation Output	35
17 Program 17 : Person Counter and Occupancy Monitoring System Using Verilog and FPGA	36
17.1 Verilog Code	36
17.2 Testbench	37
17.3 Verilog code	38
17.4 Verilog code	39
17.5 Simulation Output	40
17.6 FPGA Implementation Output	41

1 Program 1 : S-R Flip Flop

1.1 Verilog Code

```
1 module srff(q,s,r,clk,rst);
2   output q;
3   input s,r,clk,rst;
4   reg q;
5
6
7   always@(posedge clk or posedge rst)
8   begin
9       if(rst)
10          q<=1'b0;
11      else
12          begin
13              case({s,r})
14                  2'b00: q <= q;
15                  2'b01: q <= 1'b0;
16                  2'b10: q <= 1'b1;
17                  2'b11: q <= 1'bz;
18              endcase
19          end
20      end
21 endmodule
```

1.2 Testbench

```
1 module srff_tb();
2
3
4   reg s,r,clk,rst;
5   wire q;
6
7   srff f1(q,s,r,clk,rst);
8   always
9       #5 clk = ~clk;
10
11   initial
12   begin
13       clk = 0;
14       rst = 0;
15       s = 0;
16       r = 0;
17
18       #10 s = 0; r = 0;
19       #10 s = 0; r = 1;
20       #10 s = 1; r = 0;
21       #10 s = 1; r = 1;
```

```

22
23     #15 $stop;
24 end
25 endmodule

```

1.3 Simulation Output

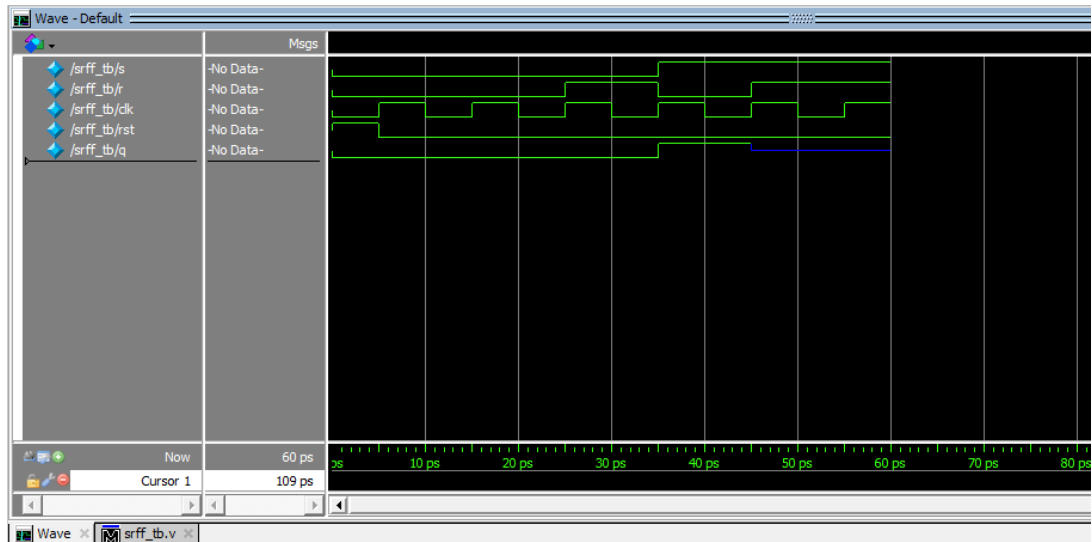


Figure 1: Simulation Output

2 Program 2 : J-K Flip Flop

2.1 Verilog Code

```
1 module jkff(q,j,k,clk,rst);
2   output q;
3   input j,k,clk,rst;
4   reg q;
5
6   always@(posedge clk or posedge rst)
7   begin
8       if(rst)
9           q<=1'b0;
10      else
11          begin
12              case({j,k})
13                  2'b00: q <= q;
14                  2'b01: q <= 1'b0;
15                  2'b10: q <= 1'b1;
16                  2'b11: q <= ~q;
17              endcase
18          end
19      end
20  end
21 endmodule
```

2.2 Testbench

```
1 module jkff_tb();
2
3   reg j,k,clk,rst;
4   wire q;
5
6   jkff f1(q,j,k,clk,rst);
7   always
8       #5 clk = ~clk;
9
10  initial
11  begin
12      clk = 0;
13      rst = 0;
14      j = 0;
15      k = 0;
16
17
18      #10 j = 0; k = 0;
19      #10 j = 0; k = 1;
20      #10 j = 1; k = 0;
21      #10 j = 0; k = 0;
```

```

22     #10 j = 0; k = 1;
23     #10 j = 1; k = 1;
24
25     #15 $stop;
26 end
27 endmodule

```

2.3 Simulation Output



Figure 2: Simulation Output

3 Program 3 : T Flip Flop

3.1 Verilog Code

```
1 module tff(q,t,rst,clk);
2 output q;
3 input t,rst,clk;
4 reg q;
5
6
7 always @(posedge clk or posedge rst)
8 begin
9     if (rst)
10         q <= 1'b1;
11     else if(t)
12         q <= ~q;
13     else
14         q <= q;
15 end
16 endmodule
```

3.2 Testbench

```
1 module tff_tb();
2
3
4 reg t,rst,clk;
5 wire q;
6
7 tff f1(q,t,rst,clk);
8 always
9     #5 clk = ~clk;
10
11 initial
12 begin
13     clk = 0;
14     rst = 1;
15     t = 0;
16     #5 rst = 0;
17     #10 t = 0;
18     #10 t = 1;
19
20     #15 $stop;
21 end
22 endmodule
```


3.3 Simulation Output

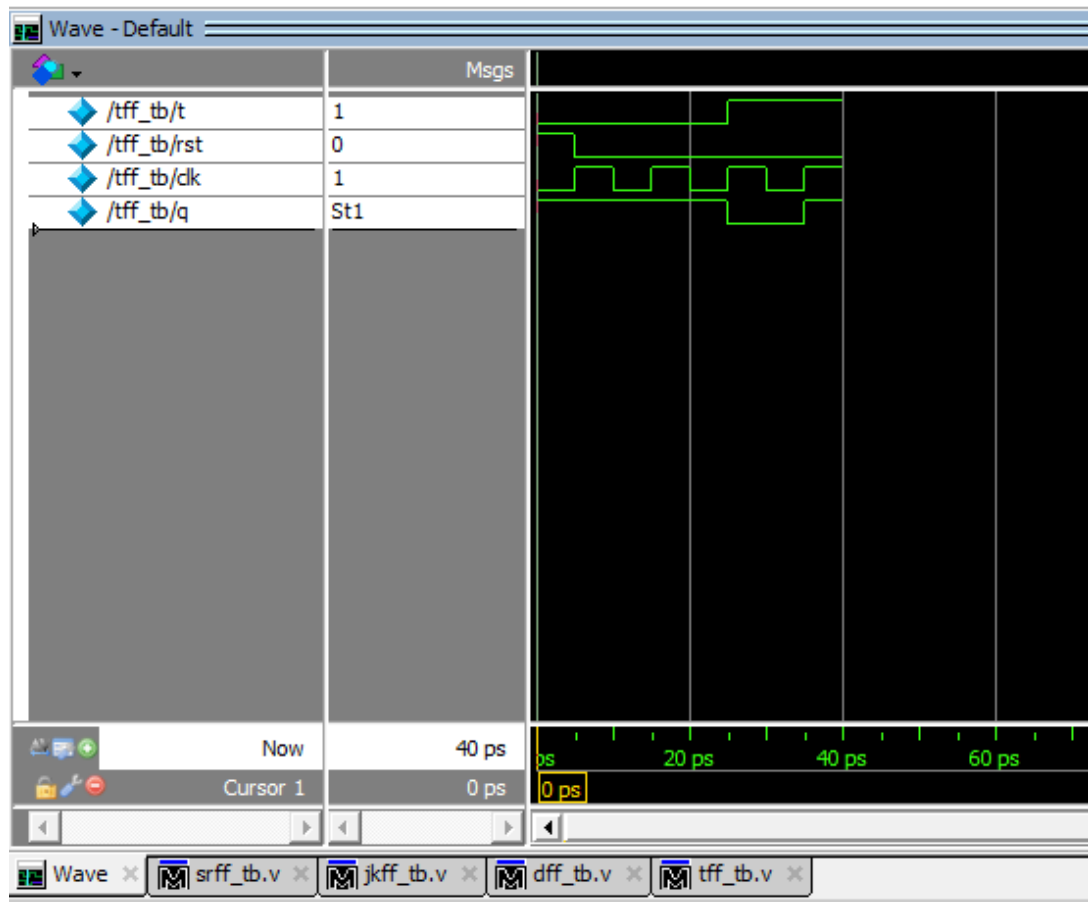


Figure 3: Simulation Output

4 Program 4 : D Flip Flop

4.1 Verilog Code

```
1 module dff(q,d,rst,clk);
2 output q;
3 input d,rst,clk;
4 reg q;
5
6
7 always @(posedge rst or posedge clk)
8 begin
9 if (rst)
10     q <= 1'b0;
11 else
12     q <= d;
13 end
14 endmodule
```

4.2 Testbench

```
1 module dff_tb();
2
3 reg d,rst,clk;
4 wire q;
5
6
7 dff f1(q,d,rst,clk);
8 always
9     #5 clk = ~clk;
10
11 initial
12 begin
13     clk = 0;
14     rst = 1;
15     d = 0;
16     #5 rst = 0;
17     #10 d = 0;
18     #10 d = 1;
19
20     #15 $stop;
21 end
22 endmodule
```

4.3 Simulation Output

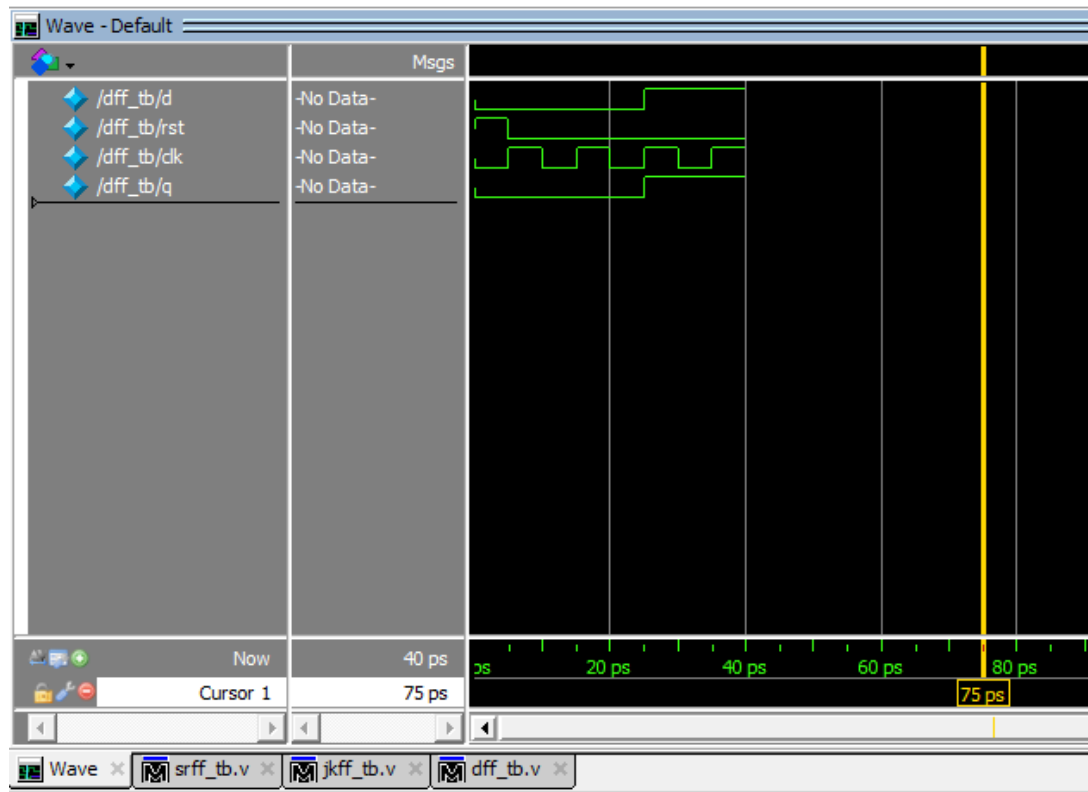


Figure 4: Simulation Output

5 Program 5 : SISO

5.1 Verilog Code

```
1
2 module siso(do,din,rst,clk);
3 output do;
4 input din,clk,rst;
5 wire q1,q2,q3;
6
7 dff f1(q3,din,rst,clk);
8 dff f2(q2,q3,rst,clk);
9 dff f3(q1,q2,rst,clk);
10 dff f4(do,q1,rst,clk);
11
12 endmodule
```

5.2 Testbench

```
1
2 module siso_tb();
3
4 reg din,clk,rst;
5 wire do;
6
7 siso s1(do,din,rst,clk);
8
9 always
10     #5 clk = ~clk;
11
12 initial
13 begin
14     din = 0;
15     clk = 0;
16     rst = 1;
17
18
19     #5 rst = 0;
20
21     #5 din = 1;
22     #10 din = 0;
23     #10 din = 1;
24     #10 din = 0;
25
26     #50 $stop;
27
28 end
29 endmodule
```

5.3 Simulation Output

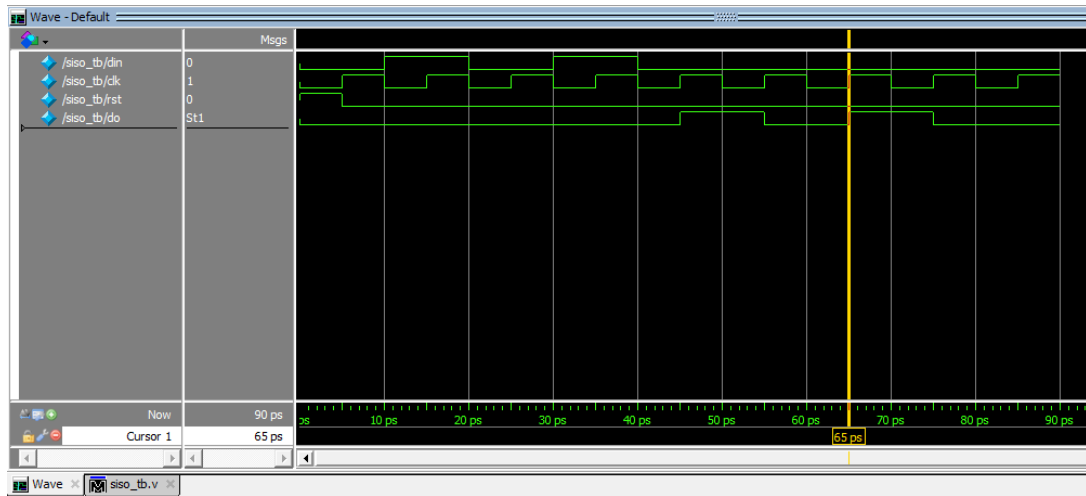


Figure 5: Simulation Output

6 Program 6 : SIPO

6.1 Verilog Code

```
1 module sipo(q,din,clk,rst);
2 output [3:0]q;
3 input din,clk,rst;
4 reg [3:0]q;
5
6
7 always@(posedge clk or posedge rst)
8
9 begin
10     if(rst)
11         q<=4'b0;
12     else
13         q<={din,q[3:1]};
14     end
15 endmodule
```

6.2 Testbench

```
1 module sipo_tb();
2
3 reg din, clk, rst;
4 wire [3:0] q;
5
6
7 sipo s1(q, din, clk, rst);
8
9 always
10     #5 clk = ~clk;
11
12 initial
13 begin
14     din = 0;
15     clk = 0;
16     rst = 1;
17
18     #5 rst = 0;
19
20     #5 din = 1;
21     #10 din = 0;
22     #10 din = 1;
23     #10 din = 1;
24
25     #50 $stop;
26
27 end
```

```

28
29 endmodule

```

6.3 Simulation Output

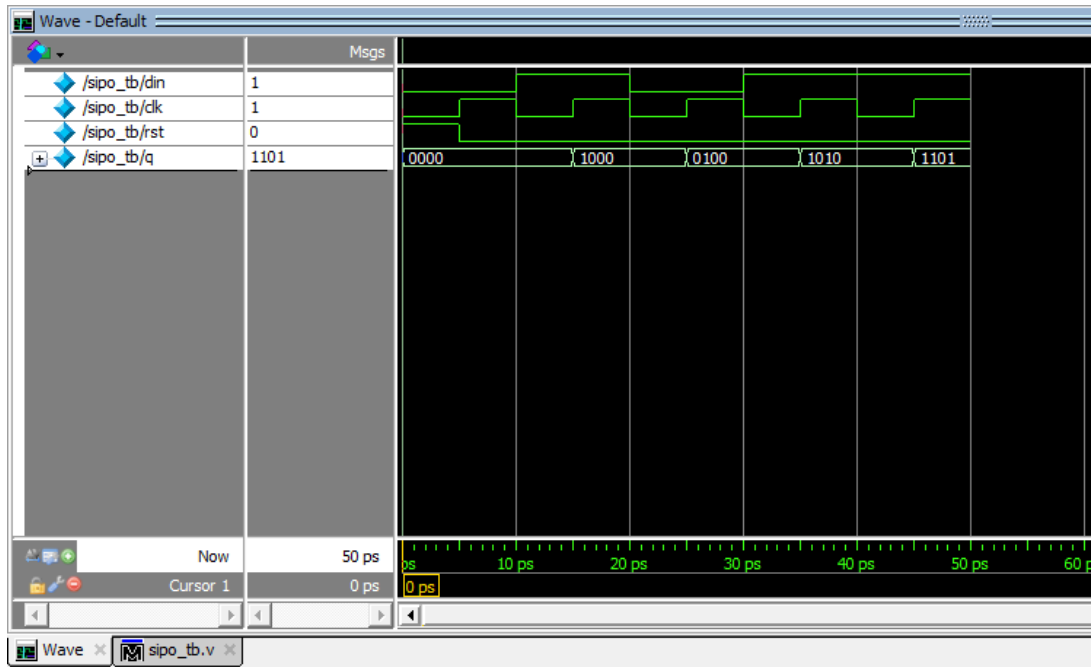


Figure 6: Simulation Output

7 Program 7 : PISO

7.1 Verilog Code

```
1 module piso(q,sout,din,load,clk,rst);
2
3
4 output [3:0] q;
5 output sout;
6
7 input [3:0] din;
8 input load,clk,rst;
9
10 reg [3:0] q;
11
12 always @(posedge clk or posedge rst)
13 begin
14     if (rst)
15         q <= 4'b0000;
16     else if (load)
17         q <= din;
18     else
19         q <= {q[2:0],1'b0};
20 end
21
22 assign sout = q[3];
23
24 endmodule
```

7.2 Testbench

```
1 module piso_tb();
2
3
4 reg [3:0] din;
5 reg load, clk, rst;
6 wire [3:0] q;
7 wire sout;
8
9 piso s1(q, sout, din, load, clk, rst);
10
11 always
12     #5 clk = ~clk;
13
14 initial
15 begin
16     din = 4'b0000;
17     load = 0;
18     clk = 0;
```



```

19     rst = 1;
20
21     #5 rst = 0;
22
23     // Parallel load 1011
24     #5  din = 4'b1011;
25         load = 1;
26
27     #10 load = 0;
28
29     // Shift
30     #10;
31     #10;
32     #10;
33     #10;
34
35     #20 $stop;
36
37 end
38
39 endmodule

```

7.3 Simulation Output

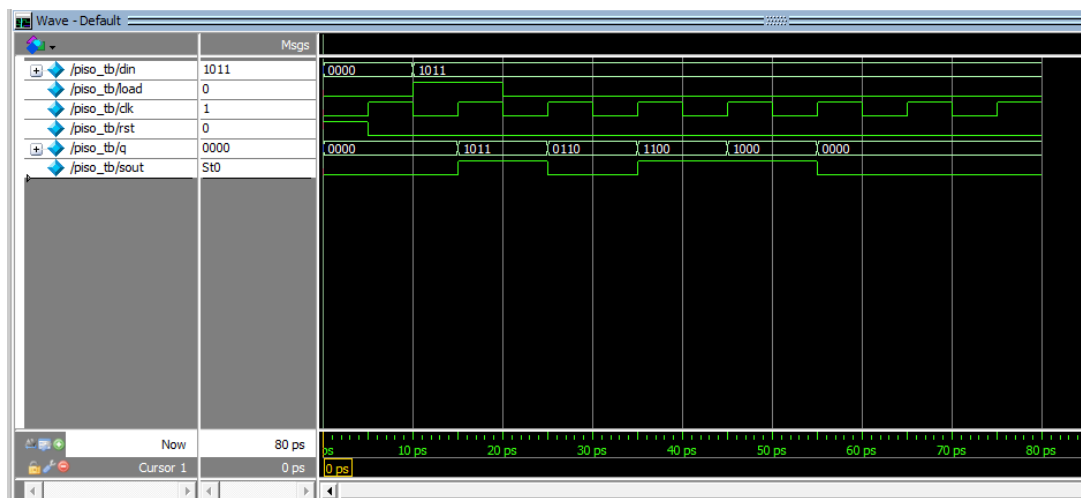


Figure 7: Simulation Output

8 Program 8 : PIPO

8.1 Verilog Code

```
1 module pipo(q,i,load,clk,rst);
2
3 output [3:0]q;
4 input [3:0]i;
5 input load,clk,rst;
6 reg [3:0]q;
7 always @(posedge clk or posedge rst)
8 begin
9   if(rst)
10    q<=4'b0;
11   else if (load)
12    q<=i;
13   else
14    q<=q;
15   end
16 endmodule
```

8.2 Testbench

```
1 module pipo_tb();
2
3 reg [3:0] i;
4 reg load, clk, rst;
5 wire [3:0] q;
6
7 pipo s1(q, i, load, clk, rst);
8
9 always
10     #5 clk = ~clk;
11
12 initial
13 begin
14     i = 4'b0000;
15     load = 0;
16     clk = 0;
17     rst = 1;
18
19     #5 rst = 0;
20
21     // Load 1011
22     #5 i = 4'b1011;
23     load = 1;
24
25     #10 load = 0;
26
```

```

27 // Load 1100
28 #10 i = 4'b1100;
29     load = 1;
30
31 #10 load = 0;
32
33 #20 $stop;
34
35 end
36
37 endmodule

```

8.3 Simulation Output

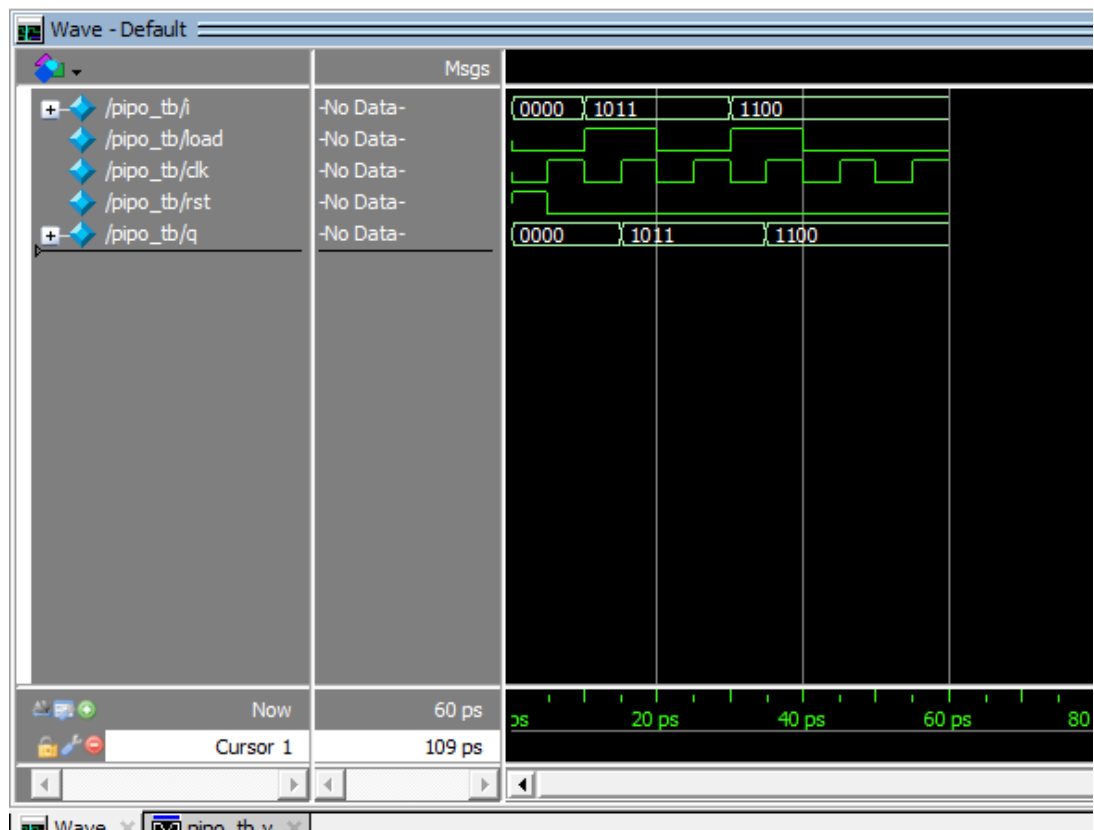


Figure 8: Simulation Output

9 Program 9 : Bidirectional Shift Register

9.1 Verilog Code

```
1 module BidirSreg(q,SDR,SDL,SIR,SIL,direction,clk,rst);
2
3 output SDR,SDL;
4 output [3:0]q;
5 input SIR,SIL;
6 input direction,clk,rst;
7 reg[3:0]q;
8
9 always@(posedge clk or posedge rst)
10
11 begin
12 if(rst)
13 q<=4'b0;
14 else if (direction)
15 q<={SIL,q[3:1]};
16 else
17 q<={q[2:0],SIR};
18 end
19 assign SDL=q[0];
20 assign SDR=q[3];
21 endmodule
```

9.2 Testbench

```
1 module BidirSreg_tb();
2
3 reg SIR, SIL;
4 reg direction, clk, rst;
5
6 wire [3:0] q;
7 wire SDR, SDL;
8
9 BidirSreg s1(q, SDR, SDL, SIR, SIL, direction, clk, rst);
10
11 always
12     #5 clk = ~clk;
13
14 initial
15 begin
16     SIR = 0;
17     SIL = 0;
18     direction = 0;
19     clk = 0;
20     rst = 1;
21
```

```

22     #5 rst = 0;
23
24     // Shift in from SIL
25     SIL = 1;
26     direction = 1;
27
28     #10 SIL = 0;
29     #10 SIL = 1;
30     #10 SIL = 1;
31
32     // Shift in from SIR
33     SIR = 0;
34     direction = 0;
35
36     #10 SIR = 0;
37     #10 SIR = 1;
38     #10 SIR = 0;
39
40     #20 $stop;
41
42 end
43
44 endmodule

```

9.3 Simulation Output

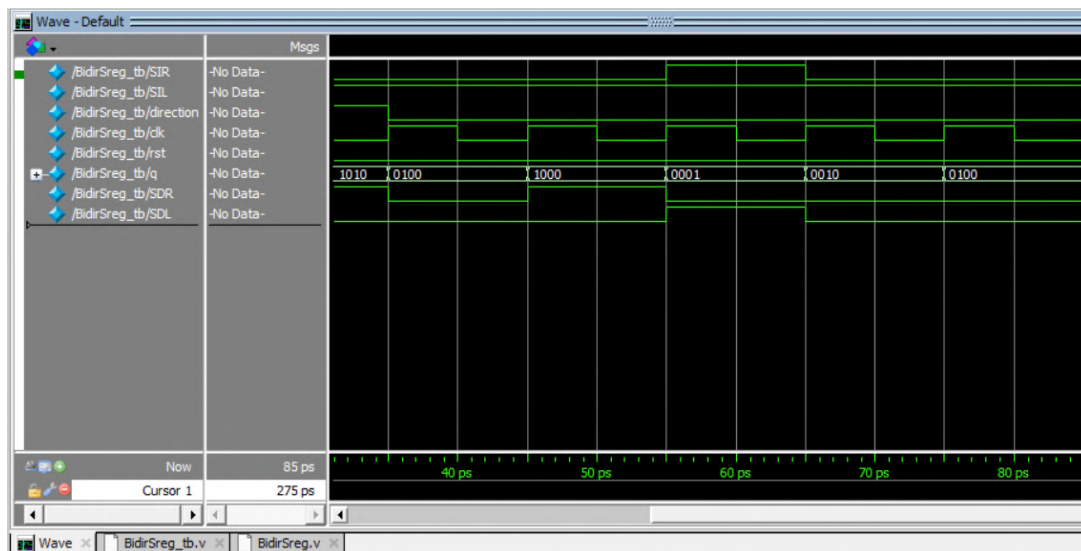


Figure 9: Simulation Output

10 Program 10 : Universal Shift Register

10.1 Verilog Code

```
1 module unvSreg(q,p,SR,SL,S,clk,rst);
2     output[3:0] q;
3     input [3:0] p;
4     input SR, SL;
5     input [1:0] S;
6     input clk;
7     input rst;
8     reg[3:0] q;
9
10    always @(posedge clk or posedge rst)
11    begin
12        if (rst)
13            q <= 4'b0000;
14
15        else
16            begin
17                case (S)
18                    2'b00: q <= q;
19                    2'b01: q <= {SR, q[3:1]};
20                    2'b10: q <= {q[2:0], SL};
21                    2'b11: q <= p;
22                endcase
23            end
24    end
25
26 endmodule
```

10.2 Testbench

```
1 module unvSreg_tb();
2
3     reg [3:0] p;
4     reg SR, SL;
5     reg [1:0] S;
6     reg clk, rst;
7
8     wire [3:0] q;
9
10    unvSreg s1(q, p, SR, SL, S, clk, rst);
11
12    always
13        #5 clk = ~clk;
14
15    initial
16    begin
```

```

17     p = 4'b0000;
18     SR = 0;
19     SL = 0;
20     S = 2'b00;
21     clk = 0;
22     rst = 1;
23
24     #5 rst = 0;
25
26     // Parallel Load: 1011
27     #5 p = 4'b1011;
28         S = 2'b11;
29
30     #10;
31
32     // Hold
33     S = 2'b00;
34
35     #10;
36
37     // Shift Right, SR = 0
38     SR = 0;
39     S = 2'b01;
40
41     #10;
42
43     // Shift Right, SR = 1
44     SR = 1;
45     S = 2'b01;
46
47     #10;
48
49     // Shift Left, SL = 0
50     SL = 0;
51     S = 2'b10;
52
53     #10;
54
55     // Shift Left, SL = 1
56     SL = 1;
57     S = 2'b10;
58
59     #20 $stop;
60
61 end
62
63 endmodule

```

10.3 Simulation Output

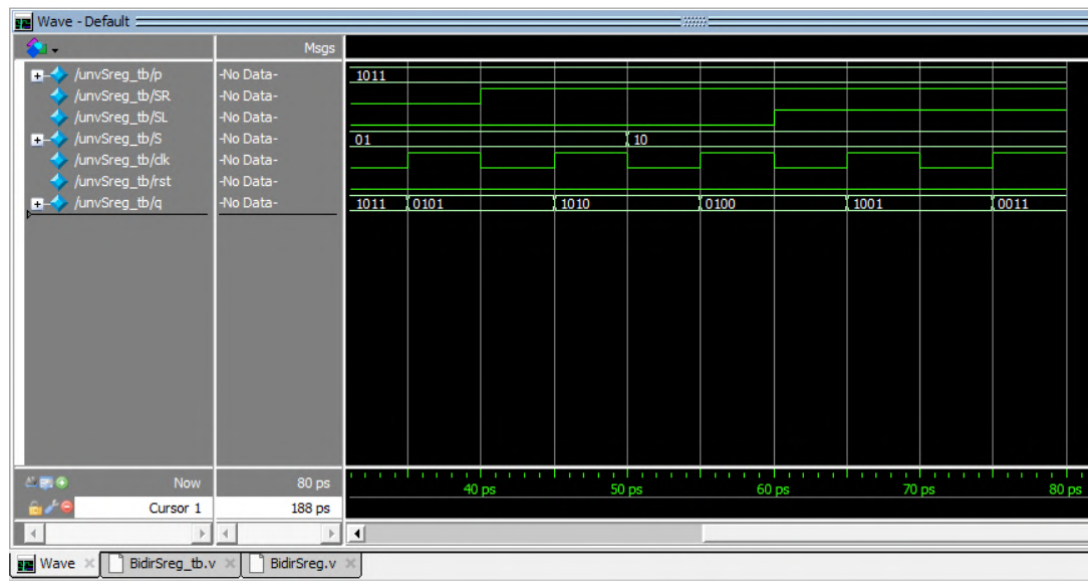


Figure 10: Simulation Output

11 Program 11 : Up Counter

11.1 Verilog Code

```
1 module upcounter(q, clk, rst);
2
3 output [3:0] q;
4 input clk, rst;
5 reg [3:0] q;
6
7 always @(posedge clk or posedge rst)
8 begin
9     if (rst)
10         q <= 4'b0000;
11     else
12         q <= q + 1'b1;
13 end
14
15 endmodule
```

11.2 Testbench

```
1 module upcounter_tb();
2
3 reg clk, rst;
4 wire [3:0] q;
5
6 upcounter s1(q, clk, rst);
7
8 always
9     #5 clk = ~clk;
10
11 initial
12 begin
13     clk = 0;
14     rst = 1;
15
16     #5 rst = 0;
17
18     #100 $stop;
19
20 end
21
22 endmodule
```

11.3 Simulation Output

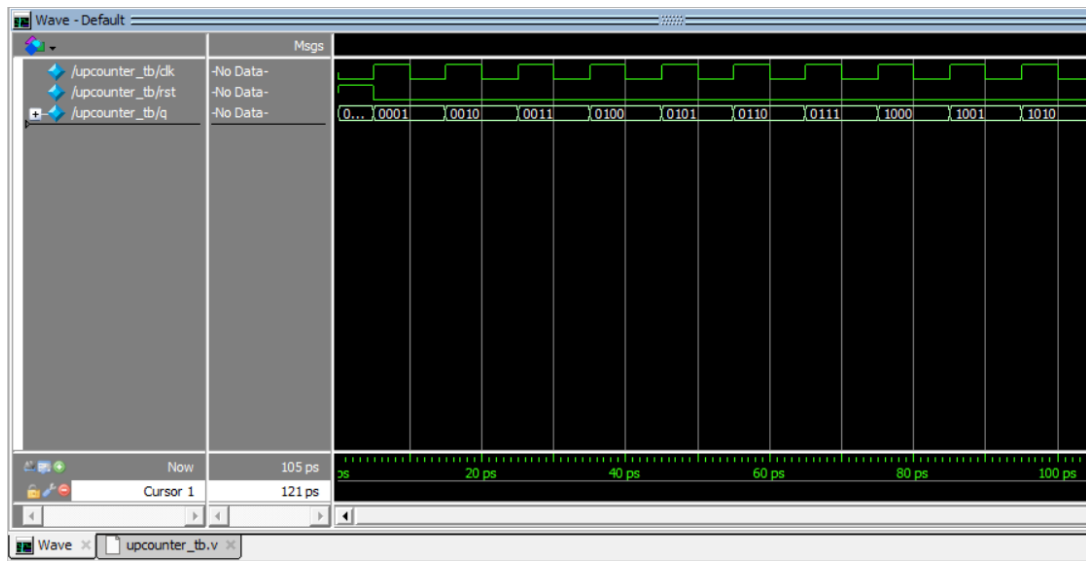


Figure 11: Simulation Output

12 Program 12 : Down Counter

12.1 Verilog Code

```
1 module downcounter(q, clk, rst);
2
3 output [3:0] q;
4 input clk, rst;
5 reg [3:0] q;
6
7 always @(posedge clk or posedge rst)
8 begin
9     if (rst)
10         q <= 4'b0000;
11     else
12         q <= q - 1'b1;
13 end
14
15 endmodule
```

12.2 Testbench

```
1 module downcounter_tb();
2
3 reg clk, rst;
4 wire [3:0] q;
5
6 downcounter s1(q, clk, rst);
7
8 always
9     #5 clk = ~clk;
10
11 initial
12 begin
13     clk = 0;
14     rst = 1;
15
16     #5 rst = 0;
17
18     #100 $stop;
19
20 end
21
22 endmodule
```

12.3 Simulation Output

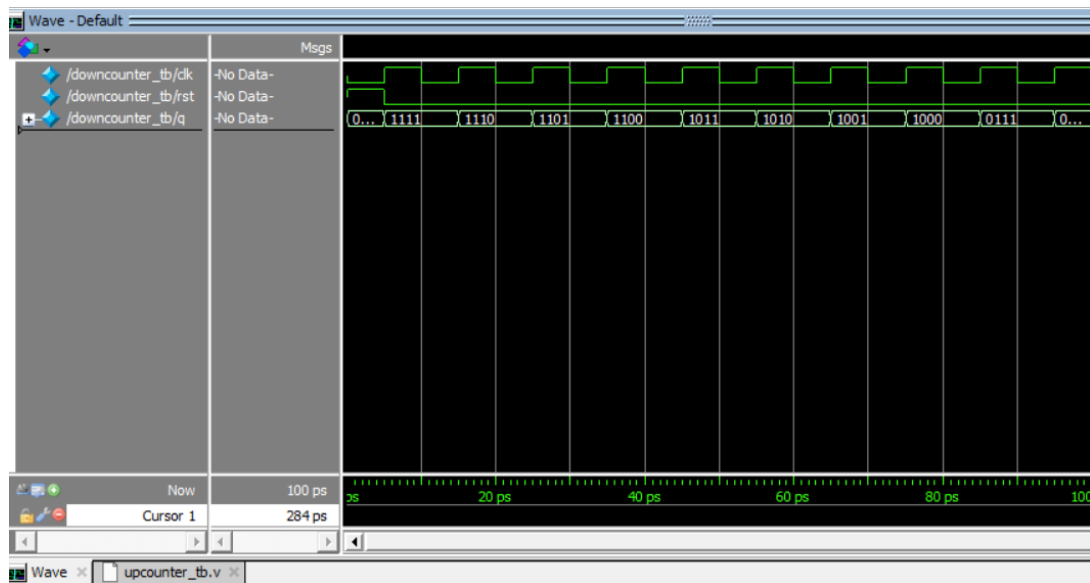


Figure 12: Simulation Output

13 Program 13 : Up-Down Counter

13.1 Verilog Code

```
1 module updown(q,s,clk, rst);
2
3 output [3:0] q;
4 input s;
5 input clk, rst;
6 reg [3:0] q;
7
8 always @(posedge clk or posedge rst)
9 begin
10     if (rst)
11         q <= 4'b0000;
12     else if(s)
13         q <= q - 1'b1;
14     else
15         q <= q + 1'b1;
16 end
17
18 endmodule
```

13.2 Testbench

```
1 module updown_tb();
2
3 reg s, clk, rst;
4 wire [3:0] q;
5
6 updown s1(q, s, clk, rst);
7
8 always
9     #5 clk = ~clk;
10
11 initial
12 begin
13     s = 0;
14     clk = 0;
15     rst = 1;
16
17     #5 rst = 0;
18
19     // UP counting
20     s = 0;
21     #50;
22
23     // DOWN counting
24     s = 1;
```

```

25     #50;
26
27     #20 $stop;
28
29 end
30
31 endmodule

```

13.3 Simulation Output

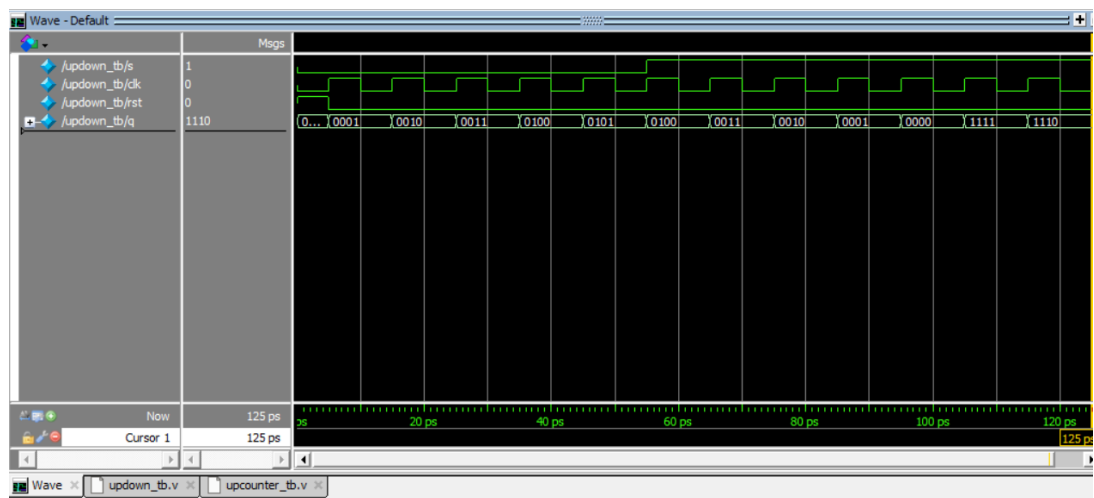


Figure 13: Simulation Output

14 Program 14 : MOD - 10 Counter

14.1 Verilog Code

```
1 module mod10count(q, clk, rst);
2
3 output [3:0] q;
4 input clk, rst;
5 reg [3:0] q;
6
7 always @(posedge clk or posedge rst)
8 begin
9     if (rst)
10         q <= 4'b0;
11     else if (q==4'b1001)
12         q <= 4'b0;
13     else
14         q <= q+1;
15 end
16
17 endmodule
```

14.2 Testbench

```
1 module mod10count_tb();
2
3 reg clk, rst;
4 wire [3:0] q;
5
6 mod10count s1(q, clk, rst);
7
8 always
9     #5 clk = ~clk;
10
11 initial
12 begin
13     clk = 0;
14     rst = 1;
15
16     #5 rst = 0;
17
18     #120 $stop;
19
20 end
21
22 endmodule
```

14.3 Simulation Output

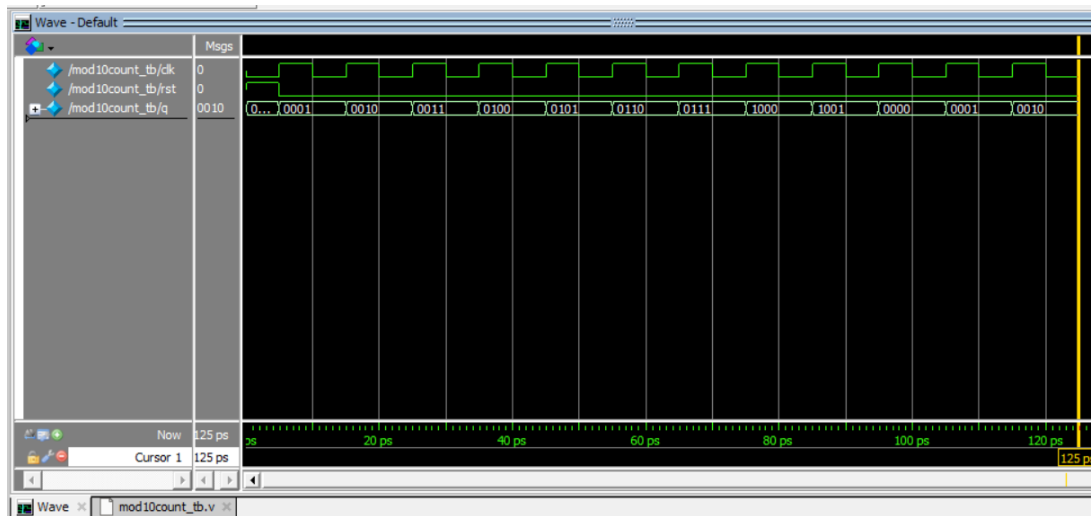


Figure 14: Simulation Output

15 Program 16 : Ring Counter

15.1 Verilog Code

```
1 module ringcount(q, clk, rst);
2
3 output [3:0] q;
4 input clk, rst;
5 reg [3:0] q;
6
7 always @(posedge clk or posedge rst)
8 begin
9     if (rst)
10         q <= 4'b1000;
11     else
12         q <={q[2:0],q[3]};
13
14 end
15
16 endmodule
```

15.2 Testbench

```
1 module ringcount_tb();
2
3 reg clk, rst;
4 wire [3:0] q;
5
6 ringcount s1(q, clk, rst);
7
8 always
9     #5 clk = ~clk;
10
11 initial
12 begin
13     clk = 0;
14     rst = 1;
15
16     #5 rst = 0;
17
18     #50 $stop;
19
20 end
21
22 endmodule
```

15.3 Simulation Output

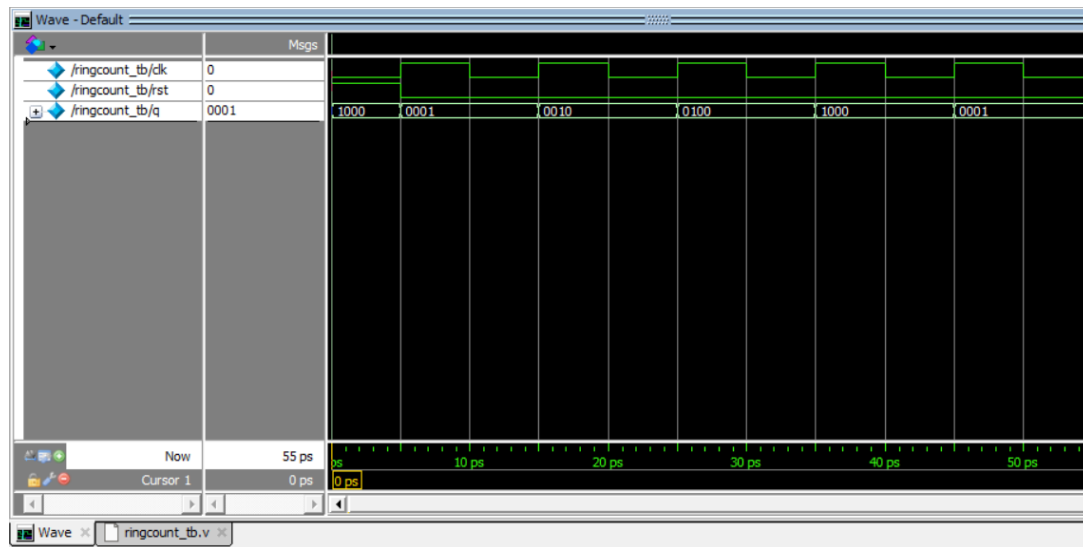


Figure 15: Simulation Output

16 Program 16 : Johnson Counter

16.1 Verilog Code

```
1 module johnsoncount(q, clk, rst);
2
3 output [3:0] q;
4 input clk, rst;
5 reg [3:0] q;
6
7 always @(posedge clk or posedge rst)
8 begin
9     if (rst)
10         q <= 4'b0000;
11     else
12         q <={q[2:0],~q[3]};
13
14 end
15
16 endmodule
```

16.2 Testbench

```
1 module johnsoncount_tb();
2
3 reg clk, rst;
4 wire [3:0] q;
5
6 johnsoncount s1(q, clk, rst);
7
8 always
9     #5 clk = ~clk;
10
11 initial
12 begin
13     clk = 0;
14     rst = 1;
15
16     #5 rst = 0;
17
18     #90 $stop;
19
20 end
21
22 endmodule
```

16.3 Simulation Output

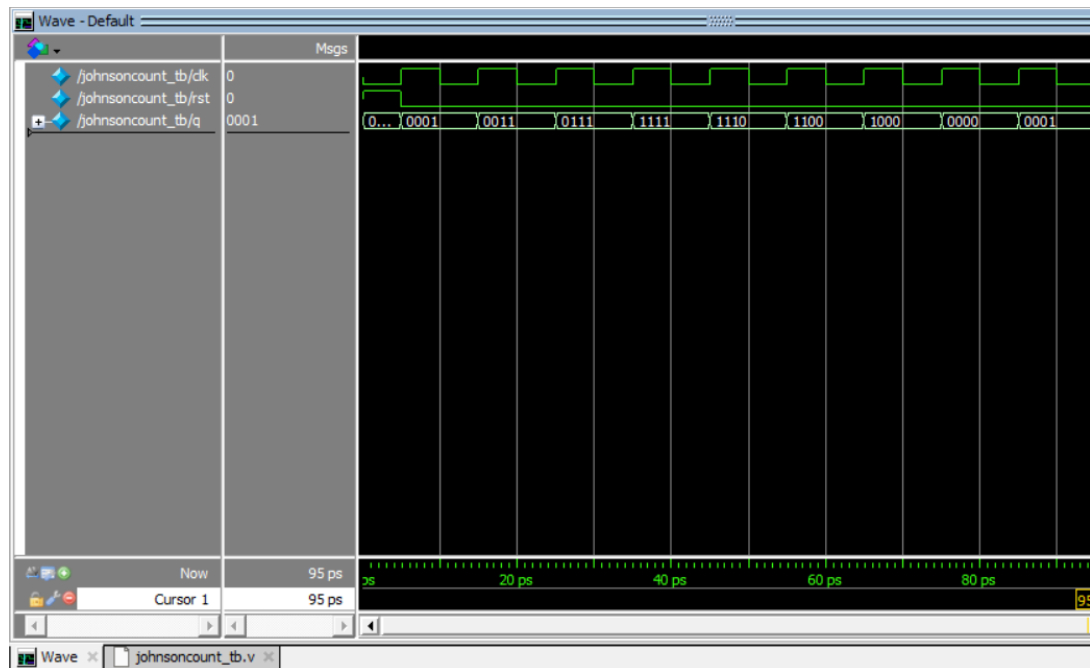


Figure 16: Simulation Output

17 Program 17 : Person Counter and Occupancy Monitoring System Using Verilog and FPGA

A company, which is putting a stall in an industrial exhibition for maximum of 99 persons, wants you to design a system, which counts the number of persons who visit the stall as well as the number of people inside the stall at any point of time. You may install lights photo-electric cells or any other sources/sensors for detecting people entering and leaving the stall. Realize the design using Verilog and display the result using seven segment display. Also write a test bench to test all possible combinations. Also verify the Verilog code using FPGA.

17.1 Verilog Code

```
1 module Assesment(peoplein,total,pin_pulse,pout_pulse,rst,clk);
2 output [6:0] peoplein,total;
3 input pin_pulse,pout_pulse,rst,clk;
4 reg [6:0] peoplein,total;
5 reg [6:0] pincount;
6 reg [6:0] poutcount;
7
8 // COUNT PEOPLE ENTERING
9
10 always @(posedge clk or posedge rst)
11 begin
12 if (rst)
13 begin
14 pincount <= 7'd0;
15 total <= 7'd0;
16 end
17 else
18 begin
19 if (pin_pulse && pincount < 7'd99)
20 pincount <= pincount + 1'b1;
21 if (pin_pulse && total < 7'd99)
22 total <= total + 1'b1;
23 end
24 end
25
26 // COUNT PEOPLE LEAVING
27
28 always @(posedge clk or posedge rst)
29 begin
30 if (rst)
31 poutcount <= 7'd0;
32 else if (pout_pulse && poutcount < pincount)
33 poutcount <= poutcount + 1'b1;
34 end
35
36 // TOTAL PEOPLE CURRENTLY INSIDE
```

```

37
38 always @(*)
39 begin
40 peoplein = pincount - poutcount;
41 end
42 endmodule

```

17.2 Testbench

```

1 module Assesment_tb();
2 reg clk,rst;
3 reg pin_pulse,pout_pulse;
4 wire [6:0] peoplein,total;
5
6 Assesment a1(peoplein,total,pin_pulse,pout_pulse,rst,clk);
7
8 initial
9 begin
10 clk = 0;
11 forever #5 clk = ~clk;
12 end
13
14 initial
15 begin
16 rst = 1;
17 pin_pulse = 0;
18 pout_pulse = 0;
19 #20 rst = 0;
20
21 // One person enters
22
23 #10 pin_pulse = 1;
24 #10 pin_pulse = 0;
25
26 // One more person enters
27
28 #10 pin_pulse = 1;
29 #10 pin_pulse = 0;
30
31 // One person leaves
32
33 #10 pout_pulse = 1;
34 #10 pout_pulse = 0;
35
36 // Entry and exit together
37
38 #10 pin_pulse = 1;
39     pout_pulse = 1;
40 #10 pin_pulse = 0;
41     pout_pulse = 0;

```

```

42
43 // Five entries
44
45 begin
46 #10 pin_pulse = 1;
47 #10 pin_pulse = 0;
48 end
49
50 // Five exits
51
52 begin
53 #10 pout_pulse = 1;
54 #10 pout_pulse = 0;
55 end
56
57 #20 $stop;
58 end
59 endmodule

```

17.3 Verilog code

```

1 module ButtonPulse(clk,rst,key,pulse);
2
3 input clk,rst,key;
4 output pulse;
5
6 reg key1,key2;
7 reg key2_prev;
8 reg pulse;
9
10 // SYNCHRONIZE AND DETECT BUTTON PRESS
11
12
13 always @(posedge clk or posedge rst)
14 begin
15     if (rst)
16     begin
17         key1 <= 1'b1;
18         key2 <= 1'b1;
19         key2_prev <= 1'b1;
20         pulse <= 1'b0;
21     end
22     else
23     begin
24         // Synchronizer
25         key1 <= key;
26         key2 <= key1;
27
28         // Previous synchronized state
29         key2_prev <= key2;

```

```

30
31         // Default: no pulse
32         pulse <= 1'b0;
33
34         // Active-low button press
35         if (key2_prev && !key2)
36             pulse <= 1'b1;
37     end
38 end
39
40 endmodule

```

17.4 Verilog code

```

1
2 module SevenSegmentCounter(CLOCK_50,KEY0,KEY1,KEY2,HEX0,HEX1,HEX2
3     ,HEX3);
4
5     input  CLOCK_50,KEY0,KEY1,KEY2;
6
7     output [6:0] HEX0,HEX1,HEX2,HEX3;
8
9     wire pin_pulse,pout_pulse;
10    wire [6:0] peoplein,total;
11
12    wire [3:0] punit,ptens,tunit,ttens;
13
14    wire rst;
15
16    assign rst = ~KEY2;
17
18    // PUSH BUTTON INPUTS
19    // KEY0 = PIN
20    // KEY1 = POUT
21    // KEY2 = RESET
22
23
24    ButtonPulse d1(CLOCK_50,rst,KEY0,pin_pulse);
25    ButtonPulse d2(CLOCK_50,rst,KEY1,pout_pulse);
26
27
28    // COUNTER
29
30
31    Assesment a1(peoplein,total,pin_pulse,pout_pulse,rst,CLOCK_50);
32
33
34    // PEOPLEIN
35

```



```

36
37 assign punit = peoplein % 10;
38 assign ptens = peoplein / 10;
39
40
41 // TOTAL
42
43
44 assign tunit = total % 10;
45 assign ttens = total / 10;
46
47
48 // SEVEN SEGMENT
49
50
51 SevenSegment s1(punit, HEX0);
52 SevenSegment s2(ptens, HEX1);
53 SevenSegment s3(tunit, HEX2);
54 SevenSegment s4(ttens, HEX3);
55
56 endmodule

```

17.5 Simulation Output

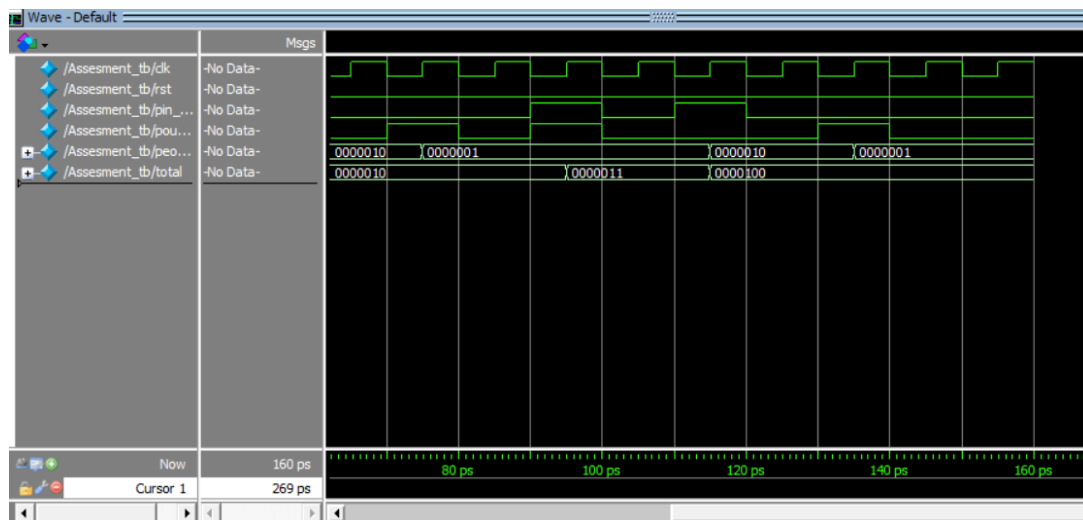


Figure 17: Simulation Output

17.6 FPGA Implementation Output



Figure 18: FPGA Implementation using push buttons

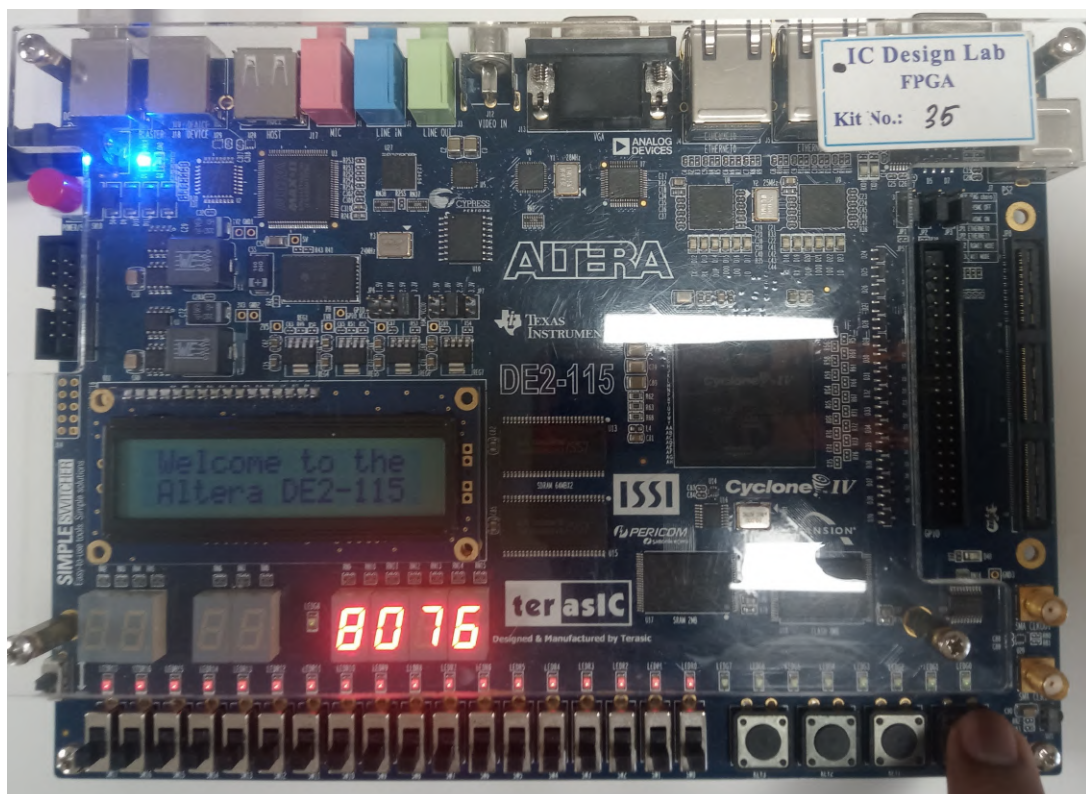


Figure 19: FPGA Implementation Output- poeple in 76 total visit 80

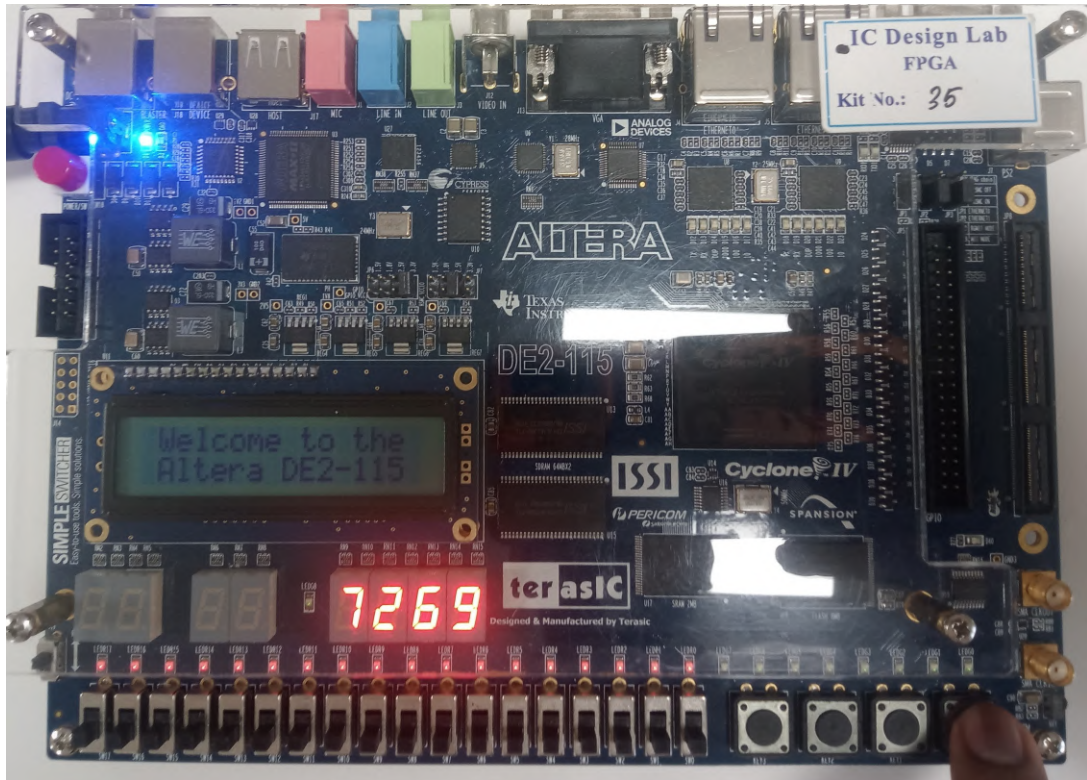


Figure 20: FPGA Implementation Output- poeple in 69 total visit 72

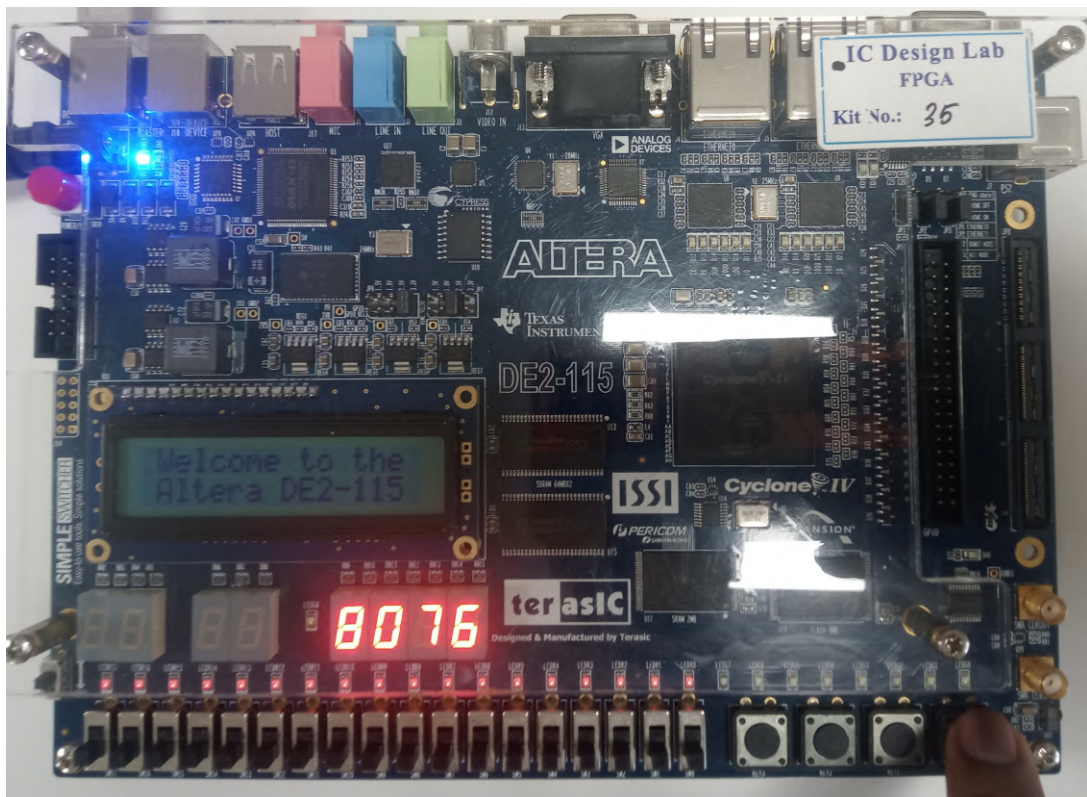


Figure 21: FPGA Implementation Output- poeple in 76 total visit 80